

WEEK-2 Practical

Task-1 Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to 3 another. Explain how control transitions between user Space and kernel space during execution.

pavani@DESKTOP-38OLEIN: ~/ossp

GNU nano 8.7.1

filecopy.c *

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead, bytesWritten;

    source = open("sample.txt", O_RDONLY);

    if (source == -1) {
        perror("Error opening source file");
        return 1;
    }

    destination = open("copy.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination == -1) {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0) {

        bytesWritten = write(destination, buffer, bytesRead);

        if (bytesWritten != bytesRead) {
            perror("Error writing to destination file");
            close(source);
            close(destination);
            return 1;
        }
    }

    if (bytesRead == -1) {
        perror("Error reading source file");
    }

    close(source);
    close(destination);

    printf("File copied successfully.\n");

    return 0;
}
```

Output:

```
pavani@DESKTOP-380LEIN:~/ossp$ gcc filecopy.c -o filecopy
pavani@DESKTOP-380LEIN:~/ossp$ echo "This is sample file content." > sample.txt
pavani@DESKTOP-380LEIN:~/ossp$ ./filecopy
File copied successfully.
pavani@DESKTOP-380LEIN:~/ossp$ cat sample.txt
txtThis is sample file content.
pavani@DESKTOP-380LEIN:~/ossp$ cat copy.txt
This is sample file content.
pavani@DESKTOP-380LEIN:~/ossp$
```

Report:

In this task, I developed a C program to copy the contents of one file into another file using the system calls `open()`, `read()`, `write()` and `close()`. The source file was opened in read mode and the destination file was opened in write mode. The contents were read into a buffer and written into the destination file. After completing the operation, both files were closed. I also studied how the control moves from user space to kernel space whenever a system call is made. The program was executed successfully and the file contents were copied correctly.

Task-2

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system 3 calls and identify the kernel services involved

```

pavani@DESKTOP-380LEIN:~/ossp$ strace -e trace=openat,read,write,close cat sample.txt
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libm.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libpcre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
read(3, "novev\tsysfs\nnovev\ttmpfs\nnovev\tbdf"..., 1024) = 442
close(3) = 0
openat(AT_FDCWD, "/proc/mounts", O_RDONLY|O_CLOEXEC) = 3
read(3, "none /usr/lib/modules/6.18.33.2-"..., 1024) = 1024
read(3, "e 0 0\ndevpts /dev/pts devpts rw,"..., 1024) = 1024
read(3, "osuid,novev,noexec,relatime,nosy"..., 1024) = 1024
read(3, "relatime 0 0\nnone /run/credentia"..., 1024) = 383
read(3, "", 1024) = 0
close(3) = 0
openat(AT_FDCWD, "/proc/self/maps", O_RDONLY|O_CLOEXEC) = 3
read(3, "5b62fa897000-5b62fa989000 r--p 0"..., 1024) = 1024
read(3, "93fe000-7c9f093ff000 r--p 000a90"..., 1024) = 1024
read(3, "00 08:30 13631"..., 1024) = 1024
read(3, " /usr/lib/x86_64-lin"..., 1024) = 1024
read(3, "0 13625 /us"..., 1024) = 566
close(3) = 0
openat(AT_FDCWD, "/usr/share/coreutils/locales/uucore/en-US.ftl", O_RDONLY|O_CLOEXEC) = 3
read(3, "# Common strings shared across a"..., 4080) = 4080
read(3, "", 32) = 0
close(3) = 0
openat(AT_FDCWD, "/usr/share/coreutils/locales/cat/en-US.ftl", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "sample.txt", O_RDONLY|O_CLOEXEC) = 3
This is sample file content.
close(5) = 0
close(4) = 0
close(3) = 0
+++ exited with 0 +++
pavani@DESKTOP-380LEIN:~/ossp$

```

Report: In this task, I used the strace utility to trace the system calls generated by the command `cat sample.txt`. The important system calls observed were `openat()`, `read()`, `write()` and `close()`. The `openat()` call was used to open the file, `read()` was used to read its contents, and `write()` displayed the contents on the terminal. Finally, `close()` released the file descriptor. From the trace, I understood how the `cat` command interacts with the kernel through system calls.